

Sistema Escolar

Parte 2: Interfaces, Clases Abstractas, MVC y Strategy

¿De dónde venimos?

En la Parte 1 construiste un sistema de gestión escolar básico usando herencia y polimorfismo. Creaste las clases Persona, Alumno, Docente y Materia, y aprendiste cómo relacionarlas entre sí. Esta segunda parte toma ese mismo sistema y lo escala incorporando cuatro conceptos fundamentales de la programación orientada a objetos:

Concepto	Para qué lo usamos
Clase Abstracta	Convertir Persona en una clase que no se puede instanciar sola, pero obliga a sus subclases a implementar ciertos métodos.
Interface	Definir contratos de comportamiento que distintas clases pueden cumplir, sin importar su jerarquía de herencia.
Patrón Strategy	Cambiar en tiempo de ejecución la forma en que se calcula el promedio de un alumno, sin modificar la clase Alumno.
Patrón MVC	Separar la lógica del sistema (Model), la presentación (View) y la coordinación (Controller) en capas bien definidas.

Objetivo de la actividad

Al terminar esta guía vas a tener un sistema que aplica todos estos conceptos de forma coherente, partiendo del código que ya escribiste en la Parte 1. Cada paso incluye la teoría necesaria, el código a escribir y un ejercicio para consolidar lo aprendido.

Repaso: Solución esperada de la Parte 1

Antes de extender el sistema, asegurate de que tu Parte 1 respeta esta estructura. Si tu código es diferente pero funciona correctamente, igual podés continuar.

Clase Persona

Persona.cs

```
public class Persona
{
    public string Nombre { get; }
    public string Email { get; }

    public Persona(string nombre, string email)
    {
        Nombre = nombre;
        Email = email;
    }

    public virtual void Presentarse()
    {
        Console.WriteLine($"Hola, soy {Nombre} ({Email}).");
    }
}
```

Clase Alumno

Alumno.cs

```
public class Alumno : Persona
{
    public int Legajo { get; }

    public Alumno(string nombre, string email, int legajo)
        : base(nombre, email)
    {
        Legajo = legajo;
    }

    public override void Presentarse()
    {
        Console.WriteLine($"Soy el alumno {Nombre}, legajo {Legajo}.");
    }
}
```

Clase Docente

Docente.cs

```
public class Docente : Persona
{
    public string NroEmpleado { get; }

    public Docente(string nombre, string email, string nroEmpleado)
        : base(nombre, email)
    {
        NroEmpleado = nroEmpleado;
    }

    public override void Presentarse()
    {
        Console.WriteLine($"Soy el docente {Nombre}, empleado N° {NroEmpleado}.");
    }
}
```

Clase Materia

Materia.cs

```
public class Materia
{
    public string Codigo { get; }
    public string Nombre { get; }
    public int Cupo { get; }
    public List<Persona> Inscriptos { get; } = new List<Persona>();

    public Materia(string codigo, string nombre, int cupo)
    {
        Codigo = codigo;
        Nombre = nombre;
        Cupo = cupo;
    }

    public void Inscribir(Persona p)
    {
        if (Inscriptos.Count < Cupo)
            Inscriptos.Add(p);
        else
            Console.WriteLine($"Sin cupo disponible en {Nombre}.");
    }
}
```

Program.cs

Program.cs

```
List<Persona> personas = new List<Persona>();

personas.Add(new Alumno("Ana", "ana@mail.com", 1001));
personas.Add(new Alumno("Bruno", "bruno@mail.com", 1002));
personas.Add(new Alumno("Carla", "carla@mail.com", 1003));
personas.Add(new Docente("Diana", "diana@mail.com", "EMP-01"));
personas.Add(new Docente("Elio", "elio@mail.com", "EMP-02"));

foreach (var p in personas)
    p.Presentarse();

// -- Materia con cupo 2 --
var mate = new Materia("MAT101", "Matemática I", 2);
mate.Inscribir(personas[0]); // Ana -> OK
mate.Inscribir(personas[1]); // Bruno -> OK
mate.Inscribir(personas[2]); // Carla -> sin cupo
```

Paso 1 — Clases Abstractas

¿Qué es una clase abstracta?

Una clase abstracta es una clase que no puede ser instanciada directamente. Existe para servir como base de otras clases, y puede exigir que sus subclasses implementen determinados métodos.

Concepto clave

Una clase abstracta puede tener:

- Métodos con implementación (igual que una clase normal).
- Métodos abstractos: solo la firma, sin cuerpo. Las subclasses DEBEN implementarlos.
- Propiedades y constructores.

En C# se declara con la palabra clave 'abstract'.

¿Por qué convertir Persona en abstracta?

En el sistema actual, Persona puede instanciarse directamente:

```
var p = new Persona("Juan", "j@mail.com"); // ¿Tiene sentido esto?
```

Pensalo: ¿existe en la realidad una persona que sea solo "Persona" y no sea ni alumno ni docente? En nuestro sistema, no. Toda persona debe ser una de sus variantes concretas. Hacer Persona abstracta refleja mejor esa realidad y previene errores.

Además, vamos a agregar el método abstracto `ObtenerRol()`, que cada subclase deberá implementar para describir su rol en el sistema.

Código: Persona como clase abstracta

Persona.cs — versión abstracta

```
public abstract class Persona
{
    public string Nombre { get; }
    public string Email { get; }

    // Constructor accesible desde subclasses
    protected Persona(string nombre, string email)
    {
        Nombre = nombre;
        Email = email;
    }

    // Método abstracto: sin cuerpo, DEBEN implementarlo las subclasses
    public abstract string ObtenerRol();

    // Método virtual: tiene implementación, las subclasses PUEDEN sobreescribirlo
}
```

```
public virtual void Presentarse()
{
    Console.WriteLine($"{ObtenerRol()} {Nombre} - {Email}");
}
}
```

Actualizar Alumno y Docente

Al hacer Persona abstracta, el compilador te va a exigir que ambas subclases implementen ObtenerRol(). Agregá el método en cada una:

Actualización de subclases

```
// En Alumno.cs - agregar:
public override string ObtenerRol() => "Alumno";

// En Docente.cs - agregar:
public override string ObtenerRol() => "Docente";
```

Atención

Si en tu Program.cs tenías 'new Persona(...)' el compilador va a marcar error. Eso es correcto: eso es exactamente lo que queremos evitar. Reemplazá esas instancias por Alumno o Docente según corresponda.

Ejercicio 1.A:

1. Modificá Persona para que sea abstract y agregá el método abstracto ObtenerRol().
2. Implementá ObtenerRol() en Alumno y Docente.
3. Actualizá Presentarse() en Persona para que use ObtenerRol() en el mensaje.
4. En Program.cs, verificá que todo compila y que Presentarse() muestra el rol.

Ejercicio 1.B:

- Intentá escribir 'var p = new Persona("X", "x@x.com");' en Program.cs.
- ¿Qué error te da el compilador? ¿Por qué tiene sentido ese error?
- Comentá la línea una vez que lo hayas observado.

Paso 2 — Interfaces

¿Qué es una interface?

Una interface define un contrato: una lista de métodos y propiedades que una clase se compromete a implementar. A diferencia de la herencia (que dice "es un tipo de"), una interface dice "puede hacer esto".

Clase abstracta vs Interface

Clase abstracta:

- Puede tener código implementado.
- Una clase solo puede heredar de UNA clase (abstracta o no).
- Modeliza una relación 'ES UN': Alumno ES UNA Persona.

Interface:

- Solo define firmas, sin implementación (en versiones modernas puede tener default, pero lo ignoramos por ahora).
- Una clase puede implementar MUCHAS interfaces.
- Modeliza una capacidad: 'PUEDE HACER': Alumno PUEDE SER Calificable.

¿Qué interfaces vamos a crear?

Vamos a definir tres interfaces que describen capacidades del sistema:

Interface	Métodos requeridos	¿Quién la implementa?
IInscribible	Inscribir(Persona)	Materia
ICalificable	AgregarNota(double) , ObtenerPromedio()	Alumno
IReportable	GenerarReporte()	Alumno, Docente, Materia

Código de las interfaces

IInscribible.cs

```
public interface IInscribible
{
    void Inscribir(Persona persona);
}
```

ICalificable.cs

```
public interface ICalificable
{
    void AgregarNota(double nota);
    double ObtenerPromedio();
}
```

IReportable.cs

```
public interface IReportable
{
    string GenerarReporte();
}
```

Actualizar Alumno para implementar ICalificable e IReportable

Alumno.cs – con interfaces

```
public class Alumno : Persona, ICalificable, IReportable
{
    public int Legajo { get; }
    private List<double> _notas = new List<double>();

    public Alumno(string nombre, string email, int legajo)
        : base(nombre, email)
    {
        Legajo = legajo;
    }

    public override string ObtenerRol() => "Alumno";

    // ICalificable
    public void AgregarNota(double nota) => _notas.Add(nota);

    public double ObtenerPromedio()
    {
        if (_notas.Count == 0) return 0;
        double suma = 0;
        foreach (var n in _notas) suma += n;
        return suma / _notas.Count;
    }

    // IReportable
    public string GenerarReporte()
    {
        return $"Alumno: {Nombre} | Legajo: {Legajo} | Promedio: {ObtenerPromedio():F2}";
    }
}
```

Actualizar Materia para implementar IInscribible e IReportable

Materia.cs – con interfaces

```
public class Materia : IInscribible, IReportable
{
    public string Codigo { get; }
    public string Nombre { get; }
    public int Cupo { get; }
```



```

public List<Persona> Inscriptos { get; } = new List<Persona>();

public Materia(string codigo, string nombre, int cupo)
{
    Codigo = codigo; Nombre = nombre; Cupo = cupo;
}

// IInscribable
public void Inscribir(Persona persona)
{
    if (Inscriptos.Count < Cupo)
        Inscriptos.Add(persona);
    else
        Console.WriteLine($"Sin cupo en {Nombre}.");
}

// IReportable
public string GenerarReporte()
{
    return $"Materia: {Nombre} ({Codigo}) | Inscriptos:
{Inscriptos.Count}/{Cupo}";
}
}

```

Ventaja de las interfaces

Con IReportable podemos hacer esto en Program.cs:

```

List<IReportable> reportables = new List<IReportable>();
reportables.Add(alumno1); // Alumno implementa IReportable
reportables.Add(materia1); // Materia implementa IReportable

foreach (var r in reportables)
    Console.WriteLine(r.GenerarReporte());

```

Aunque son tipos distintos, los tratamos de forma uniforme gracias al contrato.

Ejercicio 2.A: Implementar las interfaces

1. Creá los archivos IInscribable.cs, ICalificable.cs e IReportable.cs.
2. Actualizá Alumno para que implemente ICalificable e IReportable.
3. Actualizá Materia para que implemente IInscribable e IReportable.
4. Implementá también IReportable en Docente (mostrá nombre, email y número de empleado).

Ejercicio 2.B: Usar IReportable en Program.cs

- Creá una List<IReportable> y cargá en ella instancias de Alumno, Docente y Materia.
- Recorrela con foreach y mostrá el reporte de cada uno.
- Observá cómo el mismo código funciona para tipos completamente distintos.

Paso 3 — Patrón de Diseño: Strategy

¿Qué es el patrón Strategy?

El patrón Strategy permite definir una familia de algoritmos, encapsular cada uno y hacerlos intercambiables. El cliente (en este caso Alumno) no sabe qué algoritmo está usando, solo sabe que puede pedir el resultado.

Analogía

Pensá en el GPS de tu celular. Puede calcular la ruta más rápida, la más corta o la que evita peajes. El usuario elige la estrategia, pero el GPS simplemente ejecuta 'CalcularRuta()'. Podés cambiar la estrategia sin tocar el código del GPS en sí.

En nuestro sistema: Alumno tiene notas y pide el promedio.
La estrategia decide CÓMO se calcula ese promedio.

Estrategias de promedio que vamos a crear

Vamos a implementar tres formas de calcular el promedio de un alumno:

- PromedioSimple — suma de notas dividido cantidad (el promedio aritmético clásico).
- PromedioPonderado — las últimas notas tienen más peso que las primeras.
- PromedioSinExtremos — descarta la nota más alta y la más baja antes de promediar.

Código del patrón Strategy

IEstrategiaPromedio.cs

```
// La interface que define el contrato de la estrategia
public interface IEstrategiaPromedio
{
    double Calcular(List<double> notas);
}
```

PromedioSimple.cs

```
public class PromedioSimple : IEstrategiaPromedio
{
    public double Calcular(List<double> notas)
    {
        if (notas.Count == 0) return 0;
        double suma = 0;
        foreach (var n in notas) suma += n;
        return suma / notas.Count;
    }
}
```

PromedioPonderado.cs

```
public class PromedioPonderado : IEstrategiaPromedio
{
    // Notas más recientes tienen mayor peso
    public double Calcular(List<double> notas)
    {
        if (notas.Count == 0) return 0;
        double sumaPeso = 0;
        double totalPeso = 0;
        for (int i = 0; i < notas.Count; i++)
        {
            int peso = i + 1; // peso crece con el índice
            sumaPeso += notas[i] * peso;
            totalPeso += peso;
        }
        return sumaPeso / totalPeso;
    }
}
```

PromedioSinExtremos.cs

```
public class PromedioSinExtremos : IEstrategiaPromedio
{
    public double Calcular(List<double> notas)
    {
        if (notas.Count <= 2) return new PromedioSimple().Calcular(notas);
        var copia = new List<double>(notas);
        copia.Sort();
        copia.RemoveAt(0); // quitar la mínima
        copia.RemoveAt(copia.Count - 1); // quitar la máxima
        return new PromedioSimple().Calcular(copia);
    }
}
```

Integrar Strategy en Alumno

Ahora actualizamos Alumno para que acepte y use una estrategia de promedio. La clase ya no calcula el promedio por sí misma, lo delega a la estrategia.

Alumno.cs — con Strategy

```
public class Alumno : Persona, ICalificable, IReportable
{
    public int Legajo { get; }
    private List<double> _notas = new List<double>();

    // La estrategia de promedio (se puede cambiar en cualquier momento)
    private IEstrategiaPromedio _estrategia;

    public Alumno(string nombre, string email, int legajo,
        IEstrategiaPromedio estrategia = null)
```

```

        : base(nombre, email)
    {
        Legajo      = legajo;
        _estrategia = estrategia ?? new PromedioSimple();
    }

    // Permite cambiar la estrategia después de crear el alumno
    public void CambiarEstrategia(IEstrategiaPromedio nueva)
    {
        _estrategia = nueva;
    }

    public override string ObtenerRol() => "Alumno";

    public void AgregarNota(double nota) => _notas.Add(nota);

    // Ahora delega en la estrategia
    public double ObtenerPromedio() => _estrategia.Calcular(_notas);

    public string GenerarReporte()
    {
        return $"Alumno: {Nombre} | Legajo: {Legajo} | Promedio: {ObtenerPromedio():F2}";
    }
}

```

Usar Strategy en Program.cs

Program.cs — demo Strategy

```

var alumno = new Alumno("Ana", "ana@mail.com", 1001);
alumno.AgregarNota(4.0);
alumno.AgregarNota(7.0);
alumno.AgregarNota(9.0);
alumno.AgregarNota(6.0);

// Estrategia simple (por defecto)
Console.WriteLine("Simple:      " + alumno.ObtenerPromedio().ToString("F2"));

// Cambiar a ponderado
alumno.CambiarEstrategia(new PromedioPonderado());
Console.WriteLine("Ponderado:   " + alumno.ObtenerPromedio().ToString("F2"));

// Cambiar a sin extremos
alumno.CambiarEstrategia(new PromedioSinExtremos());
Console.WriteLine("Sin extremos: " + alumno.ObtenerPromedio().ToString("F2"));

```

Ejercicio 3.A: Implementar las estrategias

1. Creá los archivos IEstrategiaPromedio.cs, PromedioSimple.cs, PromedioPonderado.cs y PromedioSinExtremos.cs.

2. Actualizá Alumno para que use la estrategia al calcular el promedio.

3. En Program.cs, creá un alumno con 5 notas y mostrá el resultado con las 3 estrategias.

Ejercicio 3.B — DESAFÍO: Nueva estrategia: PromedioAprobacion

Creá una nueva clase PromedioAprobacion que solo incluya en el cálculo las notas ≥ 4 .

Si no hay notas aprobadas, debe devolver 0.

Probala con un alumno que tenga notas por debajo y por encima del mínimo.

Paso 4 — Patrón Arquitectónico: MVC

¿Qué es MVC?

MVC (Model – View – Controller) es un patrón que separa un sistema en tres capas con responsabilidades bien definidas. Su objetivo es que cada parte del código tenga una sola razón para cambiar.

Capa	Responsabilidad	En nuestro sistema
Model	Datos y lógica de negocio	Persona, Alumno, Docente, Materia, las interfaces y las estrategias.
View	Presentación al usuario	Una clase EscolarView que muestra menús, reportes y mensajes por consola.
Controller	Coordinación entre Model y View	Una clase EscolarController que recibe las acciones del usuario, opera sobre el Model y le pide a la View que muestre el resultado.

¿Por qué separar en capas?

Imaginate que mañana querés cambiar la aplicación de consola a una interfaz gráfica. Con MVC, solo cambiás la View. El Model y el Controller quedan igual.

O si cambian las reglas de inscripción (más cupo, validaciones nuevas), solo tocás el Model. La View no necesita saber nada de eso.

Principio: cada clase tiene UNA razón para cambiar.

Código: La View

La View solo tiene métodos para mostrar información. No toma decisiones, no accede a la base de datos, solo muestra.

EscolarView.cs

```
public class EscolarView
{
    public void MostrarMenu()
    {
        Console.WriteLine("\n===== SISTEMA ESCOLAR =====");
        Console.WriteLine("1. Listar personas");
        Console.WriteLine("2. Agregar alumno");
        Console.WriteLine("3. Agregar docente");
        Console.WriteLine("4. Inscribir alumno en materia");
        Console.WriteLine("5. Ver reporte de materia");
        Console.WriteLine("6. Ver promedio de alumno");
        Console.WriteLine("0. Salir");
        Console.Write("Opcion: ");
    }
}
```

```

public void MostrarMensaje(string mensaje)
{
    Console.WriteLine(mensaje);
}

public void MostrarPersonas(List<Persona> personas)
{
    Console.WriteLine("\n-- Personas en el sistema --");
    foreach (var p in personas)
        p.Presentarse();
}

public void MostrarReporte(IReportable reportable)
{
    Console.WriteLine(reportable.GenerarReporte());
}

public string PedirTexto(string prompt)
{
    Console.Write(prompt);
    return Console.ReadLine();
}

public int PedirEntero(string prompt)
{
    Console.Write(prompt);
    return int.Parse(Console.ReadLine());
}
}

```

Código: El Model (repositorio)

Para que el Controller pueda operar sobre los datos sin conocer los detalles de almacenamiento, creamos una clase que actúa como repositorio central del sistema:

EscolarModel.cs

```

public class EscolarModel
{
    public List<Persona> Personas { get; } = new List<Persona>();
    public List<Materia> Materias { get; } = new List<Materia>();

    public void AgregarPersona(Persona p) => Personas.Add(p);
    public void AgregarMateria(Materia m) => Materias.Add(m);

    public Alumno BuscarAlumnoPorLegajo(int legajo)
    {
        foreach (var p in Personas)
            if (p is Alumno a && a.Legajo == legajo) return a;
        return null;
    }
}

```

```

    }

    public Materia BuscarMateriaPorCodigo(string codigo)
    {
        foreach (var m in Materias)
            if (m.Codigo == codigo) return m;
        return null;
    }
}

```

Código: El Controller

El Controller recibe las acciones del usuario, accede al Model y le indica a la View qué mostrar. Es el intermediario.

EscolarController.cs

```

public class EscolarController
{
    private readonly EscolarModel _model;
    private readonly EscolarView _view;

    public EscolarController(EscolarModel model, EscolarView view)
    {
        _model = model;
        _view = view;
    }

    public void Ejecutar()
    {
        bool continuar = true;
        while (continuar)
        {
            _view.MostrarMenu();
            string opcion = _view.PedirTexto("");
            switch (opcion)
            {
                case "1": _view.MostrarPersonas(_model.Personas); break;
                case "2": AgregarAlumno(); break;
                case "3": AgregarDocente(); break;
                case "4": InscribirAlumno(); break;
                case "5": VerReporteMateria(); break;
                case "6": VerPromedioAlumno(); break;
                case "0": continuar = false; break;
                default: _view.MostrarMensaje("Opcion invalida."); break;
            }
        }
    }

    private void AgregarAlumno()
    {

```



```
        string nombre = _view.PedirTexto("Nombre: ");
        string email = _view.PedirTexto("Email: ");
        int legajo = _view.PedirEntero("Legajo: ");
        _model.AgregarPersona(new Alumno(nombre, email, legajo));
        _view.MostrarMensaje("Alumno agregado.");
    }

    private void AgregarDocente()
    {
        string nombre = _view.PedirTexto("Nombre: ");
        string email = _view.PedirTexto("Email: ");
        string nro = _view.PedirTexto("Nro empleado: ");
        _model.AgregarPersona(new Docente(nombre, email, nro));
        _view.MostrarMensaje("Docente agregado.");
    }

    private void InscribirAlumno()
    {
        int legajo = _view.PedirEntero("Legajo del alumno: ");
        string codigo = _view.PedirTexto("Codigo de materia: ");
        var alumno = _model.BuscarAlumnoPorLegajo(legajo);
        var materia = _model.BuscarMateriaPorCodigo(codigo);
        if (alumno == null || materia == null)
        {
            _view.MostrarMensaje("Alumno o materia no encontrado.");
            return;
        }
        materia.Inscribir(alumno);
        _view.MostrarMensaje($"Alumno inscripto en {materia.Nombre}.");
    }

    private void VerReporteMateria()
    {
        string codigo = _view.PedirTexto("Codigo de materia: ");
        var materia = _model.BuscarMateriaPorCodigo(codigo);
        if (materia == null)
            _view.MostrarMensaje("Materia no encontrada.");
        else
            _view.MostrarReporte(materia);
    }

    private void VerPromedioAlumno()
    {
        int legajo = _view.PedirEntero("Legajo: ");
        var alumno = _model.BuscarAlumnoPorLegajo(legajo);
        if (alumno == null)
            _view.MostrarMensaje("Alumno no encontrado.");
        else
            _view.MostrarReporte(alumno);
    }
}
```

Nuevo Program.cs con MVC

Program.cs — versión MVC

```
var model      = new EscolarModel();
var view       = new EscolarView();
var controller = new EscolarController(model, view);

// Cargar datos de prueba
model.AgregarPersona(new Alumno("Ana", "ana@mail.com", 1001));
model.AgregarPersona(new Alumno("Bruno", "bruno@mail.com", 1002));
model.AgregarMateria(new Materia("MAT101", "Matematica I", 3));

// Ejecutar el sistema
controller.Ejecutar();
```

Punto importante

Observá que Program.cs ahora no contiene ninguna lógica de negocio. Solo crea las tres partes del MVC y dispara la ejecución. Toda la lógica está distribuida correctamente en Model, View y Controller.

Ejercicio 4.A: Implementar el MVC

1. Creá los archivos EscolarModel.cs, EscolarView.cs y EscolarController.cs.
2. Actualizá Program.cs para que use el patrón MVC.
3. Probá el sistema: agregá alumnos, inscribí en materias y consultá reportes.

Ejercicio 4.B — DESAFÍO: Agregar opción al menú

Agregá al menú la opción '7. Cambiar estrategia de promedio'.

El Controller debe pedir el legajo del alumno y la estrategia deseada (1=simple, 2=ponderado, 3=sin extremos),

luego llamar a CambiarEstrategia() y mostrar el nuevo promedio.

Pensá en qué capa debe ir cada parte de esta nueva funcionalidad.

Resumen: Estructura final del proyecto

Tu solución debe tener la siguiente estructura de archivos:

Archivo	Capa	Descripción
<code>Persona.cs</code>	Model	Clase abstracta base
<code>Alumno.cs</code>	Model	Hereda Persona, implementa ICalificable, IReportable, usa Strategy
<code>Docente.cs</code>	Model	Hereda Persona, implementa IReportable
<code>Materia.cs</code>	Model	Implementa IInscribible, IReportable
<code>IInscribible.cs</code>	Interfaces	Contrato de inscripción
<code>ICalificable.cs</code>	Interfaces	Contrato de calificación
<code>IReportable.cs</code>	Interfaces	Contrato de reporte
<code>IEstrategiaPromedio.cs</code>	Strategy	Interface del patrón Strategy
<code>PromedioSimple.cs</code>	Strategy	Estrategia: promedio aritmético
<code>PromedioPonderado.cs</code>	Strategy	Estrategia: notas recientes con más peso
<code>PromedioSinExtremos.cs</code>	Strategy	Estrategia: descarta extremos
<code>EscolarModel.cs</code>	MVC - Model	Repositorio central de datos
<code>EscolarView.cs</code>	MVC - View	Presentación en consola
<code>EscolarController.cs</code>	MVC - Ctrl	Coordinación de acciones
<code>Program.cs</code>	Entrada	Ensambla MVC y arranca el sistema

Felicitaciones

Si llegaste hasta acá, aplicaste cuatro de los conceptos más importantes de la programación orientada a objetos en un sistema cohesivo y funcionando.

Clases abstractas, interfaces, Strategy y MVC son los mismos patrones que vas a encontrar en sistemas reales: frameworks web, aplicaciones de escritorio y APIs.

La diferencia con este ejercicio es solo la escala, no los conceptos.

Ahora puedes seguir aprendiendo más patrones.